SpecialistRoute Handover — 7. Known Issues and Gotchas

Oct 25, 2026 · @Anas Alsirafy

One issue needs checking before anything else: the fine-tuning callback very likely never unfreezes the backbone under Keras 3. Everything else below is a solved problem, recorded so it is not hit again.

Top risk: fine-tuning probably never happened

`FineTuneCallback` (Main section 5) sets the backbone to trainable in the middle of `model.fit` without recompiling. Keras 3 keeps the trainable settings a model had when it was compiled, so the change is ignored: only the learning-rate drop takes effect.

- **Reproduced outside the project** (TensorFlow 2.21, Keras 3.15) with the same callback logic: backbone weights changed by exactly 0.0 in every epoch after "unfreezing". Training in two stages (fit, unfreeze, recompile, fit again) did change them.
- **Recompiling inside the callback is not a fix:** it crashes Keras 3 (`'NoneType' object is not callable`).
- **Affected if confirmed:** Classifier, Bell pepper, Corn, Potato and Grape (presets with fine-tune = True) were really trained with a frozen backbone. Their results stay valid, but the paper must say "frozen backbone, learning rate lowered at epoch X". Tomato, Apple and Rest are unaffected.
- **Consistent with what we saw:** fine-tuned components were often selected before their unfreeze epoch.

Check on the real checkpoints (Main, after sections 1–8):

```
import numpy as np
for name, p in COMPONENTS.items():
    if not p.get("fine_tune"):
        continue
    r = next(x for x in json.load(open(progress_path(name)))
["per_seed_results"] if x["seed"] == 42)
    m = tf.keras.models.load_model(component_checkpoint(name, 42))
    base = next(l for l in m.layers if hasattr(l, "layers"))
    ctor, wkey = {"convnextbase": (tf.keras.applications.ConvNeXtBase,
"convnextbase_imagenet_weights"),
                  "ConvNeXtLarge": (tf.keras.applications.ConvNeXtLarge,
"convnextlarge_imagenet_weights")}[p["model"]]
    ref = ctor(include_top=False, weights=MODEL_PATHS[wkey], input_shape=
(p["img_size"], p["img_size"], 3))
    pairs = [(a, b) for a, b in zip(base.get_weights(), ref.get_weights()) if
a.shape == b.shape]
    same = sum(np.array_equal(a, b) for a, b in pairs)
    print(f"{name:12s} selected epoch {r['selected_epoch']:>3} (unfreeze at
{p['fine_tune_at_epoch']}) | "
          f"backbone tensors identical to ImageNet: {same}/{len(pairs)}")
    del m, ref; tf.keras.backend.clear_session(); gc.collect()
```

A component selected **after** its unfreeze epoch whose backbone is still 100% identical to ImageNet confirms the bug. A component selected before unfreezing proves nothing either way.

Fix: train in two stages in `train_single_split`:

1. `fit(epochs=fine_tune_at_epoch)` with the backbone frozen.
2. Unfreeze, then `compile_model(model, fine_tune_lr)`.
3. `fit(initial_epoch=fine_tune_at_epoch, epochs=epochs)`.

Keep the stage-1 best val_loss as the checkpoint's `initial_value_threshold`, set the CSV logger to `append=True` for stage 2, and join the two histories. Optionally stop each stage on a validation plateau (early-stopping patience about 25, LR patience about 10).

Gotchas

Symptom	Cause	Fix / status
Pipeline accuracy 8 points lower than expected (v1: 75.93% vs 84.26%)	Evaluation loaded images with PIL bicubic; training used TensorFlow bilinear	Fixed: TensorFlow loader (`_load_tf_image`) everywhere

Symptom	Cause	Fix / status
Router predictions mapped to the wrong specialist	Class order taken from config keys, not from the training folder	Fixed: class order read from the folder (`component_class_names`)
Potato training stuck at loss ≈ 0.69 (ln 2), 50% accuracy	EfficientNet-B0 Noisy Student weights silently skipped under Keras 3 (`skip_mismatch`)	Fixed: Potato uses ConvNeXt-Base (Corn preset)
`load_model` fails on GMP models	Custom layer not registered	Import `gmp_layer` before loading
Out of memory after several models in one kernel	TensorFlow does not return GPU memory within a process	Fixed: section 11a-2 runs each model in its own subprocess, checkpointed, retrying at a smaller batch
`check_protocol` flags ~211 "shared" files	Kaggle reuses file names across splits	Fixed: leakage check compares MD5 content
`STALE folder` error	Source dataset changed after a derived folder was built	Delete the named derived folder and re-run
`FileExistsError` when saving	A checkpoint with that name exists	By design; change `experiment_name` or let the runner move the half-finished file aside
Old results quietly moved	Preset changed; orchestrator archived them	By design: see `Experiments/v2_superseded/` and `SUPERSEDED.txt`
CLAHE fidelity check returns `inf`	Data-prep `img_size` set to 384	Use 224
`verify_clahe_dataset.py` flags every image	"Original" folder pointed at already-processed images	Point it at the raw Kaggle images
`to_excel` positional-argument error	pandas API change	Use `to_excel(writer, sheet_name=...)`

Symptom	Cause	Fix / status
`pip install` refused ("externally managed")	Ran against system Python	Install into `tf_env` (Environment Setup)
Metrics reflect the last epoch, not the best	`ModelCheckpoint` saves the best model but does not restore it in memory	Runner reloads the checkpoint from disk before scoring
ConvNeXt results unchanged with or without `preprocess_input`	It is a no-op; normalisation is inside the model	Not a bug
Archive notebook breaks Main	Overlapping names	Never run `PlantDoc_Archive_Tools.ipynb` in the same kernel
WSL / VS Code disconnects (error 1006)	WSL deactivated	`wsl --shutdown`, reopen, reconnect

Limitations to disclose

- **Presets came from v1,** where configurations were compared on the test set, so they keep a small amount of that optimism.
- **Potato's backbone changed** from EfficientNet-B0 Noisy Student (484 px) to ConvNeXt-Base (384 px) because the original weights do not load under Keras 3.
- **Fine-tuning:** components flagged as fine-tuned may in fact have frozen backbones (top risk above). Confirm before writing the methods section.
- **Small validation sets** (10–59 images per specialist): checkpoint selection and design choices are noisy at that size. Seed-to-seed spread in Tomato's validation loss alone is about ± 0.03.
- **Validation is easier than test:** the test set was never cleaned, so it is a slightly different distribution.
- **9 duplicate groups touching the test set** were flagged but kept (`reports/duplicate_flags.csv`).
- **The ensemble needs 5× the inference** of a single model.
- **Look-alike grouping in the label-quality folds is unconfirmed:** every component reported 0 pairs grouped, which may be a file-name matching problem. Verify before reporting those scores.